Data Science (DATA11001)

Technical Report

Sami Heinimo, Sandra Sillanpää & Mari Karlsson

We wanted to make a sensible and easy to use application, which would benefit both us and a wide user base. At first, we thought about making a survey about student lunch place preferences and collecting answers from students in Helsinki. But, after planning, we noticed that even if we could spread the survey on various platforms, the number of responses would probably still be low. This could cause biases in the data, and the outcome wouldn't be reliable. After careful re-planning, we ended up with our current project.

We started building a backend by creating our own database filled with food recipes. Each one of us created a text file containing multiple recipes, which we would later process. We also made separate scripts to convert text files to json-files, since they are easier to handle in data processing.

We had decided to include search functionality in the application, which could find all the recipes which included the ingredient in the search. We wanted the search to display all possible recipes despite the inflected form of an ingredient word, which meant that we needed to save all the base forms on ingredient names to database. To enable this, we decided to create three database tables:

Table "*recipes*" including the recipe name, id number to recipe and instructions.

Table "*ingredients*" including the base form of ingredient and id number for it.

Table "*recipe_ingredients*", junction table which combines the recipe id number and the ingredients needed in the recipe. It also contains values amount and unit (for ingredients).

At this point, we also implemented Python Poetry to manage dependencies.

Before creating tables, we needed to process the recipe data we had collected. We started by tokenizing all the ingredients. In this phase, file `tokenizing.py` handles the ingredients by lowering the letters, and with imported Python library called Regular Expression syntax (`re`),

removing brackets and text inside them, removing size describing words, removing extra spaces, and in some cases, removing brand names. After this, the remaining words will be split into so-called tokens. These tokens are saved in the list form and can be used later. File `tokenizing.py` also combines our separate json files to one shared `recipes_tokenized.json` file, which facilitates later processing of the data.

As tokenizing was done, the next step was to process tokens to base forms, alias to lemmatize them. This happens in `lemmatizing.py`, which uses imported library Voikko. Voikko is "a morphological analyzer, spelling and grammar checker, hyphenator and collection of related linguistic data for Finnish language" [1]. Since our recipes were in Finnis, Voikko was a suitable choice for our project. Voikko analyzes the tokens (ingredient words), and constructs a comprehensive dictionary, including keys like "baseform", "class", "number" and "wordbases" for one token. This analysis was added to `recipes_tokenized.json` file so that we had concentrated all the processed and needed data in one file.

After processing all the ingredients through tokenizing and lemmatizing, we noticed that analysis included several versions of base forms for some tokens. In most cases, multiple versions were caused by inflected form in the ingredient list, which had several possibilities for base word (see Image 1). To avoid multiple or incorrect types of tokens in the database, we decided to filter these processed tokens before adding them into the database. First, we only wanted to include words which had class "noun" and were in singular form. We defined this in file `import_recipes.py` by going through analysis dictionary (made by Voikko) and filtering tokens from there. File `import_recipes.py` also checked existing relations between recipes and ingredients to avoid adding one ingredient multiple times in the *recipe_ingredient* table.

```
"token": "voita",
"analysis": [
  {
    "BASEFORM": "voittaa",
    "CLASS": "teonsana",
    "FSTOUTPUT": "[Lt][Xp]voittaa[X]voita[Tk][Ap][P2][Ny][Eb]",
    "MOOD": "imperative",
    "NEGATIVE": "both",
    "NUMBER": "singular",
    "PERSON": "2",
    "STRUCTURE": "=ppppp",
    "TENSE": "present_simple",
    "WORDBASES": "+voittaa(voittaa)"
  },
  {
    "BASEFORM": "vuo",
    "CLASS": "nimisana",
    "FSTOUTPUT": "[Ln][Xp]vuo[X]vo[Sp][Nm]ita",
    "NUMBER": "plural",
    "SIJAMUOTO": "osanto",
    "STRUCTURE": "=ppppp",
    "WORDBASES": "+vuo(vuo)"
  },
  {
    "BASEFORM": "voi",
    "CLASS": "nimisana",
    "FSTOUTPUT": "[Ln][Xp]voi[X]vo[Sp][Nm]ita",
    "NUMBER": "plural",
    "SIJAMUOTO": "osanto",
    "STRUCTURE": "=ppppp",
    "WORDBASES": "+voi(voi)"
  },
  {
    "BASEFORM": "voi",
    "CLASS": "nimisana",
    "FSTOUTPUT": "[Ln][Xp]voi[X]voi[Sp][Ny]ta",
    "NUMBER": "singular",
    "SIJAMUOTO": "osanto",
    "STRUCTURE": "=ppppp",
    "WORDBASES": "+voi(voi)"
  }
```


Image 1. Several possible base forms for ingredient “voi” (butter).

Next, we set up our database in the file `database.py`. This file defines the database settings and creates the database engine and sessions needed to connect and communicate with the database using Python toolkit SQLAlchemy. After this, we could actually build the database.

We wanted the database to contain the tables mentioned above (*recipes, ingredients* and *recipe_ingredients*). We created `models.py` which defined the structure of database tables, containing columns for each table we have already decided to include. In `models.py`, tables are defined as classes, and there is key – value pairs mapped to certain columns in the database. Relationships between tables are defined using SQLAlchemy’s `relationship()` -function. File `create_tables.py` uses `import_recipes.py` and `models.py` to construct PSQL database with Aiven cloud data platform.

After setting up the database and importing recipes and their ingredients to it, we had access to see what the database actually looked like. Table *ingredients* seemed odd, since some of the ingredients had several various base forms saved (from only one ingredient), and some of the ingredients weren't saved at all since they were originally recognized as plural and were then filtered off. We tried again, this time only filtering the tokens by class "noun" and taking the singular form filter off.

Plural ingredients were now included, but duplicates caused by several various base forms (which all were nouns) stayed. This was a problem since our application was supposed to display ingredient names for each recipe straight from *ingredients* database table. The application wouldn't be very readable or even usable if the ingredient list in recipes had several unnecessary "ingredients". The best example for this: the original recipe said "paksuja pekoninviipaleita tai silavaa, ilman nahkaa ja hienoksi kuutioituna". Displayed ingredients for this were "pekoniviipale" (bacon), "ilma" (air) and "nahka" (skin).

So, filtering still needed some fixing. We discussed using an external library to identify the category of the token and filter all but food words off with it. However, fixing this problem succeeded without the library. We edited file `import_recipes.py` to filter the tokens and only take into account the tokens which were nouns, get base form from these tokens, and choose first base form from Voikko's analysis dictionary (it here was more than one identified base form for the token). This enabled that recipes in the application could be displayed in readable form.

Since some of the tokens were unclear or identified incorrectly in Voikko's analysis, we created separate file `ingredient_overrides.py` to take these exceptions into account and to simplify the processing of tokens. Some amount or unit values were also saved incorrectly in the database. Since these were single cases, we converted analyzed tokens into Pandas-frame in file `recipes_tokenized_to_csv.py` and saved them into `reseptit.csv` file. This enabled us to check incorrect values and correct them by hand. After correcting values, we updated the database to have the corrected values so that the recipe displayed to the user would be clearer.

We needed to make some changes to our initial (coding) plans on the way to achieve our planned application. Keeping the project canvas in our minds, we adapted our work so that the

functionality would match our original plans. Some of the tasks during the project (like choosing right base form from ingredients) took more time than we thought, but with successful planning and division of work we managed to overcome these kinds of challenges. Also, some challenges were solved with the help of AI tools, since using them was allowed in this course. Using AI tools not only made working quicker but also helped with problems which we couldn't solve even with the three of us together. Nevertheless, we stayed on track of what was happening in our code and why.

References:

[1] Voikko, 2019. [Online]. [Cited 14.10.2026] Available: https://voikko.puimula.org/